

ARRISE — Dual Monetization Technical Design Document (TDD)

This document serves as the comprehensive technical blueprint for building the ARRISE backend system. It outlines a **Dual Monetization Strategy** supporting both B2C Subscriptions (Education) and B2B Pay-Per-View Campaigns (Advertising), optimized for a **Zero-Cost Hostinger Deployment**.

1. Business Model Overview

Track 1: Subscription (B2C — Education)

- **Target:** Students, Schools, Teachers.
- **Model:** Monthly/Yearly recurring revenue.
- **Flow:** Student logs in → Server verifies active subscription → AR content unlocks.
- **Tiers:**
 - **Free:** ₹0 (e.g., 3 scans/day max).
 - **Student:** ₹99/month (Unlimited scans).
 - **School:** ₹4,999/month (Bulk accounts, school dashboard).

Track 2: View-Based (B2B — Clients)

- **Target:** Brands, Advertising Agencies.
- **Model:** Pay-per-scan (Pre-paid view buckets).
- **Flow:** Public user scans client logo → No login required → Server decrements campaign view count → AR content loads.
- **Tiers:**
 - **Starter:** ₹5,000 for 1,000 views.
 - **Growth:** ₹25,000 for 10,000 views.

2. Infrastructure & Hosting Architecture

To avoid the massive bandwidth costs associated with Firebase or AWS when hosting heavy Unity WebGL files, this architecture uses a **Split Deployment Strategy**.

The "Split" Node.js + Hostinger Architecture (Recommended)

Component	Technology	Hosting Provider	Monthly Cost
Frontend (WebAR)	HTML, JS, Unity WebGL	Hostinger (Shared Plan)	₹0 (Uses existing plan, unlimited bandwidth)
Backend API	Node.js + Express.js	Render / Railway / Vercel	₹0 (Free Tier)
Database	MySQL (or PostgreSQL)	Hostinger DB / Supabase	₹0 (Included in Hostinger / Free Tier)

Component	Technology	Hosting Provider	Monthly Cost
Authentication	JWT (JSON Web Tokens)	Handled by Backend API	₹0
Payments	Razorpay APIs + Webhooks	External Service	~2% per transaction

Workflow: The heavy `Build/` folder and `index.html` sit on Hostinger. When `index.html` loads, it executes lightweight `fetch()` requests to `api.yourdomain.com` (hosted on Render) to verify users or track views before initializing the Unity Engine.

3. Database Schema (SQL/Relational)

If using MySQL (via Hostinger) or PostgreSQL, here is the exact table structure required.

Table: `users`

Stores all student, school admin, and client credentials.

```
CREATE TABLE users (  
  id VARCHAR(50) PRIMARY KEY, -- e.g., 'usr_12345'  
  email VARCHAR(255) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL, -- bcrypt hash  
  role ENUM('student', 'school_admin', 'client', 'superadmin') NOT NULL  
  DEFAULT 'student',  
  plan_tier ENUM('free', 'student', 'school', 'client_starter',  
  'client_growth') DEFAULT 'free',  
  subscription_status ENUM('active', 'expired', 'cancelled', 'none')  
  DEFAULT 'none',  
  subscription_expiry DATETIME NULL,  
  school_id VARCHAR(50) NULL, -- Foreign key if belonging to a school  
  daily_scans INT DEFAULT 0,  
  last_scan_date DATE NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Table: `campaigns`

Stores B2B client advertising campaigns.

```
CREATE TABLE campaigns (  
  id VARCHAR(50) PRIMARY KEY, -- e.g., 'camp_abc'  
  client_id VARCHAR(50) NOT NULL, -- FK to users.id  
  campaign_name VARCHAR(255) NOT NULL,  
  target_image_key VARCHAR(100) UNIQUE NOT NULL, -- Matches Unity image  
  target name  
  views_used INT DEFAULT 0,  
  views_limit INT NOT NULL,
```

```

    status ENUM('active', 'paused', 'exhausted', 'expired') DEFAULT
    'active',
    start_date DATETIME NOT NULL,
    end_date DATETIME NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (client_id) REFERENCES users(id)
);

```

Table: `scan_logs` (Analytics)

Records every successful AR scan for billing and client dashboards. *Note: This table grows massive. Index heavily on `campaign_id` and `user_id`.*

```

CREATE TABLE scan_logs (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id VARCHAR(50) NULL, -- Null if it's a public client campaign
  campaign_id VARCHAR(50) NULL, -- Null if it's a student subscription
  scan
  target_image_key VARCHAR(100) NOT NULL,
  device_os VARCHAR(50) NULL, -- e.g., 'iOS 16', 'Android 13'
  location_country VARCHAR(50) NULL,
  scanned_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

4. API Endpoints Specification (Node.js/Express)

The backend developer needs to expose the following RESTful API endpoints.

Authentication & Users

- `POST /api/auth/register`
 - **Body:** { email, password, role }
 - **Action:** Hashes password with bcrypt, saves to DB, returns JWT.
- `POST /api/auth/login`
 - **Body:** { email, password }
 - **Action:** Verifies hash, returns JWT (expires in 30 days).

AR Gatekeeping (Called by `index.html`)

- `GET /api/ar/check-subscription`
 - **Headers:** Authorization: Bearer <JWT_TOKEN>
 - **Action:** Checks `users` table for `subscription_status == 'active'`. If 'free', checks if `daily_scans < 3`.
 - **Returns:** { allowed: true, plan: 'student', scansRemaining: 999 }
- `POST /api/ar/track-campaign-view`
 - **Body:** { target_image_key, device_os } (No JWT required, public endpoint).

- **Action:** Finds campaign by `target_image_key`. If `views_used < views_limit` and date is valid, increments `views_used` by 1 and inserts into `scan_logs`.
 - **Returns:** `{ allowed: true }` or `{ allowed: false, error: 'Campaign Exhausted' }`.
-

5. Razorpay Integration Flow (Payments)

Handling recurring subscriptions (B2C) and one-time campaign payments (B2B).

Step 1: Order Creation

When a user clicks "Subscribe" on your website:

1. Frontend calls your Backend: `POST /api/payments/create-order`
2. Backend calls Razorpay API: `razorpay.orders.create({ amount: 9900, currency: "INR" })` (*Amount is in paise*).
3. Backend returns the `order_id` to the Frontend.

Step 2: Checkout

1. Frontend opens the Razorpay Checkout UI using the `order_id`.
2. User pays via UPI/Card.

Step 3: Webhook Verification (CRITICAL)

Never trust the frontend to confirm a payment.

1. Razorpay sends an automatic POST request (Webhook) to your server: `POST /api/webhooks/razorpay`
2. The payload contains `event: payment.captured`.
3. Your Backend uses the Razorpay Secret Key to verify the webhook signature.
4. If valid, your Backend updates the database:

```
UPDATE users SET subscription_status = 'active', subscription_expiry =  
DATE_ADD(NOW(), INTERVAL 1 MONTH) WHERE id = 'usr_123';
```

6. Frontend Interception (Unity WebGL `index.html`)

The logic inside `index.html` must intercept the user *before* `createUnityInstance` runs.

```
// Example logic for index.html  
async function checkAccessAndStartAR(campaignId = null) {  
  // 1. IS THIS A CLIENT CAMPAIGN? (Public Access)  
  if (campaignId) {  
    const res = await fetch('https://api.rionick.com/api/ar/track-  
campaign-view', {
```

```

        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ target_image_key: campaignId,
device_os: navigator.platform })
    });
    const data = await res.json();
    if (data.allowed) {
        StartAR(); // Initialize Unity
    } else {
        document.getElementById('error-screen').innerText = "This AR
Campaign has ended.";
    }
    return;
}

// 2. IS THIS A STUDENT SUBSCRIPTION? (Requires Login)
const token = localStorage.getItem('arrise_jwt_token');
if (!token) {
    window.location.href = "/login.html"; // Redirect to login
    return;
}

const res = await fetch('https://api.rionick.com/api/ar/check-
subscription', {
    headers: { 'Authorization': `Bearer ${token}` }
});
const data = await res.json();

if (data.allowed) {
    StartAR(); // Initialize Unity
} else {
    window.location.href = "/upgrade.html"; // Redirect to payment
}
}

```

7. Implementation Roadmap for Developer

1. **Sprint 1 (Backend Core):** Set up Node/Express, connect to MySQL, create User/Campaign tables, build JWT Auth endpoints.
2. **Sprint 2 (AR Gating):** Build the `/check-subscription` and `/track-campaign-view` endpoints. Integrate the JavaScript `fetch` logic into the Unity `index.html`.
3. **Sprint 3 (Payments):** Integrate Razorpay Orders API and configure the Webhook endpoint to update SQL tables upon successful payment.
4. **Sprint 4 (Dashboards):** Build the HTML/React dashboards for Clients (to see views) and Admins (to manage subscriptions).